Week 4 ST2 Report

Task 1

Results of a player winning:

```
normal
You selected:  normal
Player health: 60
Enemy health: 30

ENEMY TURN
Enemy selected:  heal
Player health: 60
Enemy health: 45

PLAYER TURN
Select move...
normal
You selected:  normal
Player health: 60
Enemy health: 25

ENEMY TURN
Enemy selected:  last stand
Player health: 30
Enemy health: 10

PLAYER TURN
Select move...
norma;
Invalid move. Try again.
Select move...
normal
You selected:  normal
Player health: 30
Enemy health: -10

GAME OVER: Player wins.
```

These results were obtained when using the move normal throughout the RPG game.

Throughout this playthrough, I managed to increase my health to well over 130 points. And then fought the CPU (Enemy) to the end with a draw in result, final score of Player 0 points and CPU -5 points.

```
Player health: 55
Enemy health: 35

ENEMY TURN
Enemy selected:  normal
Player health: 35
Enemy health: 35

PLAYER TURN
Select move...
normal
You selected:  normal
Player health: 35
Enemy health: 15

ENEMY TURN
Enemy selected:  special
Player health: 25
Enemy health: 20

PLAYER TURN
Select move...
special
You selected:  special
Player health: 30
Enemy health: 10

ENEMY TURN
Enemy selected:  last stand
Player health: 0
Enemy health: -5

DRAW.
```

After 5 trys, I could not lose the game even when attempting different strategies on the CPU.

Task 2

Linear and Binary Search Algorithm:

Target: 500000

```
Linear search execution time: 1.506600 seconds
Binary search execution time: 0.000303 seconds
```

Target: 100

```
Linear search execution time: 0.000292 seconds
Binary search execution time: 0.000296 seconds
```

Target: 80000

```
Linear search execution time: 0.241894 seconds
Binary search execution time: 0.000304 seconds
```

Comparison:

For larger data, Linear search was highly ineffective. Would never use for large data structures, however binary search was the winner in all fields with its efficiency and quick time finding its target. This is due to the methodology of each algorithm, linear searches time grows linearly $O(n)$, whilst binary searches time grows algorithmically $O(\log n)$. Essentially halves the work load each step to find the specific value it is looking for instead of looking through everything.

Task 3

Numbers between [1-1000] with range [10000]:

```
Bubble Sort Execution Time: 3.852723 seconds
Insertion Sort Execution Time: 1.664297 seconds
```

Numbers between [1-1000] with range [15000]:

```
Bubble Sort Execution Time: 8.673052 seconds
Insertion Sort Execution Time: 3.725057 seconds
```

Numbers between [1-1000] with range [20000]:

```
Bubble Sort Execution Time: 15.491446 seconds
Insertion Sort Execution Time: 6.576788 seconds
```

Comparison:

Bubble sort on all three occasions did not go well to the increasing amount of data in each test, whilst the Insertion sort did tremendously well to the increase of data in comparison to the Bubble Sort. This is due to the performance killer of swapping the numbers around which uses three techniques and one temporary variable. Whilst that, insertion instead “shifts” the data instead of using multiple techniques to sort it. By picking up the element to be inserted, creates a "hole”, and slides the larger elements over by one position until the right spot is found.

Task 4

Test 1: Data size range [10000]

```
Insertion Sort + Linear Search Execution Time: 1.612708 seconds
Bubble Sort + Linear Search Execution Time: 3.798229 seconds
Insertion Sort + Binary Search Execution Time: 1.611746 seconds
Bubble Sort + Binary Search Execution Time: 3.790260 seconds
```

Test 2: Data size range [20000]

```
Insertion Sort + Linear Search Execution Time: 6.490098 seconds
Bubble Sort + Linear Search Execution Time: 15.123864 seconds
Insertion Sort + Binary Search Execution Time: 6.767145 seconds
Bubble Sort + Binary Search Execution Time: 15.324638 seconds
```

Test 3: Data size range [50000]

```
Insertion Sort + Linear Search Execution Time: 40.705595 seconds
Bubble Sort + Linear Search Execution Time: 97.046465 seconds
Insertion Sort + Binary Search Execution Time: 40.687449 seconds
Bubble Sort + Binary Search Execution Time: 96.931747 seconds
```

Comparison:

The most interesting observation is the massive increase in execution time when we add a sorting step. Task 2 (Search Only): The linear search on 1,000,000 items took roughly 1.5 seconds. Task 4 (Sort + Search): For a much smaller dataset (50,000 items), the total time jumped to 40.7 seconds (Insertion) and 97 seconds (Bubble). So, if we are searching for an item once, it is significantly faster to use a "slow" Linear Search on unsorted data than it is to sort the data first just to use a "fast" Binary Search. The "sorting tax" is simply too high for a single query.

Why Insertion Sort + Binary Search is better?

Even though both are $O(n^2)$, the results show Bubble Sort taking more than double the time of Insertion Sort at a data size of 50000. This is because Bubble Sort performs constant swaps, while Insertion Sort performs shifts as mentioned earlier. This reduction in memory operations makes a massive difference as n increases.